

# Semantic Codebase Search — Detailed Beginner Notes

## 1. Introduction

### What Are We Building?

We are building an AI-powered search system that can understand the *meaning* of code.

Instead of matching exact words, the system understands semantic similarity.

Example:

User query:

```
Where is user authentication handled?
```

Code:

```
verify_access_token(token)
```

Even though the words are different, the system understands that:

- authentication
- token verification
- login validation
- auth middleware

are related concepts.

This is called:

## Semantic Search

---

## 2. Why This Project Is Important

This project teaches many real-world AI engineering concepts:

| Concept             | Why Important                          |
|---------------------|----------------------------------------|
| Embeddings          | Foundation of modern AI retrieval      |
| Vector Databases    | Used in GenAI systems                  |
| RAG                 | Core architecture behind AI assistants |
| Retrieval Pipelines | Used in enterprise AI                  |
| Chunking            | Critical for AI search systems         |
| Backend APIs        | Required for real applications         |
| Code Parsing        | Important for developer tools          |
| AI System Design    | Valuable engineering skill             |

This project is similar to technologies used in:

- GitHub Copilot
- Cursor
- Sourcegraph
- Continue.dev

---

## 3. High-Level Architecture

```
Repository Files
  ↓
Read Code Files
  ↓
Chunk Code
  ↓
Generate Embeddings
  ↓
Store in Vector Database
  ↓
User Query
  ↓
Convert Query to Embedding
  ↓
Similarity Search
  ↓
Return Relevant Code
```

## 4. Important Concepts

### 4.1 What Is Semantic Search?

Traditional search:

```
authentication == authentication
```

Semantic search:

```
Does this text MEAN something similar?
```

Example:

| Query              | Matching Code         |
|--------------------|-----------------------|
| login verification | verify_token()        |
| user auth          | validate_jwt()        |
| payment processing | process_transaction() |

The AI understands meaning instead of exact words.

---

### 4.2 What Are Embeddings?

Embeddings are numerical representations of meaning.

Example:

```
"authentication"  
→ [0.22, -0.44, 0.91, ...]
```

```
"verify token"  
→ [0.20, -0.41, 0.89, ...]
```

These vectors become mathematically close because the meanings are related.

---

## 4.3 Why Vectors?

AI models cannot directly understand raw text.

So text is converted into:

numbers → vectors → embeddings

Then mathematics is used to compare meanings.

## 4.4 What Is Cosine Similarity?

Cosine similarity measures:

How similar are two vectors?

Values:

| Value | Meaning      |
|-------|--------------|
| 1     | Very similar |
| 0     | Unrelated    |
| -1    | Opposite     |

Visualization:

authentication ↗  
verify token ↗  
  
similar direction  
→ high similarity

## 4.5 What Is a Vector Database?

A vector database stores embeddings and performs similarity search.

Instead of:

Find exact words

it does:

Find nearest vectors

Popular vector databases:

| Tool     | Type                        |
|----------|-----------------------------|
| FAISS    | Local vector search         |
| ChromaDB | Beginner-friendly vector DB |
| Pinecone | Cloud vector DB             |
| Weaviate | Production vector DB        |

For beginners:

## Start with FAISS

### 4.6 What Is Chunking?

Large files are difficult for AI systems.

Instead of embedding entire files:

```
# BAD
whole_file_embedding
```

we split code into smaller pieces.

Example chunks:

```
def login_user():
```

```
def verify_token():
```

These smaller units are called:

## chunks

---

### 4.7 Why Chunking Matters

Good chunking improves:

- search accuracy
- retrieval quality
- AI explanations
- context relevance

Poor chunking leads to:

- irrelevant results
- noisy retrieval
- weak AI answers

Chunking is one of the most important parts of RAG systems.

---

### 4.8 What Is RAG?

RAG = Retrieval Augmented Generation

Pipeline:

```
User Question
    ↓
Retrieve Relevant Chunks
    ↓
Send Chunks to LLM
    ↓
Generate Answer
```

Example:

User:

How does authentication work?

System: 1. retrieves relevant authentication functions 2. sends them to LLM 3. LLM explains the flow

This is how many AI assistants work.

---

## 4.9 What Is AST?

AST = Abstract Syntax Tree

It converts code into structured representations.

Example:

```
def login():
```

becomes:

Function Definition

instead of plain text.

---

## 4.10 Why AST Matters

Without AST:

Chunk = random lines

With AST:

Chunk = actual function/class

This dramatically improves semantic retrieval.

---

## 5. Technology Stack

### Beginner-Friendly Stack

| Component     | Technology            |
|---------------|-----------------------|
| Language      | Python                |
| Embeddings    | Sentence Transformers |
| Vector Search | FAISS                 |
| Backend API   | FastAPI               |
| Frontend      | Streamlit             |
| Parsing       | Python AST            |
| Optional LLM  | OpenAI or Ollama      |

---

## 6. Project Roadmap

### Phase 1 — Learn Embeddings

Goal:

Build semantic text search.

Do NOT start with code search immediately.

---

### Step 1 — Install Libraries

```
pip install sentence-transformers
pip install numpy
pip install scikit-learn
```

## Step 2 — Generate Embeddings

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

embedding = model.encode("authentication system")

print(embedding)
```

## Understanding This Code

### SentenceTransformer

Loads a pretrained embedding model.

### all-MiniLM-L6-v2

A lightweight embedding model.

It converts text into vectors.

### encode()

Converts text into embeddings.

## Step 3 — Build Tiny Semantic Search

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer('all-MiniLM-L6-v2')

documents = [
    "user authentication system",
    "payment gateway integration",
```

```
    "database connection logic",  
    "JWT token verification"  
]  
  
embeddings = model.encode(documents)  
  
query = "login validation"  
  
query_embedding = model.encode([query])  
  
scores = cosine_similarity(query_embedding, embeddings)  
  
best_index = scores.argmax()  
  
print(documents[best_index])
```

---

## What You Learn Here

| Concept             | Learned |
|---------------------|---------|
| Embeddings          | Yes     |
| Semantic similarity | Yes     |
| Cosine similarity   | Yes     |
| Retrieval basics    | Yes     |

---

## Phase 2 — Learn Vector Search

### Step 1 — Install FAISS

```
pip install faiss-cpu
```

---

### Step 2 — Create Vector Index

```
import faiss  
import numpy as np
```

```
vectors = np.array(embeddings).astype("float32")

dimension = vectors.shape[1]

index = faiss.IndexFlatL2(dimension)

index.add(vectors)
```

---

## Understanding This

### IndexFlatL2

Creates a similarity search index.

---

### add()

Stores embeddings inside the vector database.

---

## Step 3 — Search Vectors

```
query_embedding = model.encode([query]).astype("float32")

D, I = index.search(query_embedding, k=2)

print(I)
```

---

## Understanding Search Output

### D

Distance scores.

---

# I

Indexes of matching vectors.

---

## Phase 3 — Read Real Files

Goal:

Search actual files instead of hardcoded text.

---

### Step 1 — Read Files

```
import os

for root, dirs, files in os.walk("repo"):
    for file in files:
        if file.endswith(".py"):
            path = os.path.join(root, file)

            with open(path, "r", encoding="utf-8") as f:
                content = f.read()

            print(content)
```

---

## What You Learn

| Concept       | Purpose               |
|---------------|-----------------------|
| os.walk()     | Traverse directories  |
| file handling | Read repository files |
| filtering     | Select Python files   |

---

## Phase 4 — Chunk Code

Goal:

Split code into meaningful pieces.

---

## Beginner Chunking

```
chunks = content.split("\n\n")
```

This is naive chunking.

Good enough initially.

---

## Better Chunking Later

Use:

- AST
- Tree-sitter

to extract:

- functions
  - classes
  - methods
- 

## Example AST Parsing

```
import ast

with open("auth.py", "r") as f:
    tree = ast.parse(f.read())

for node in ast.walk(tree):
    if isinstance(node, ast.FunctionDef):
        print(node.name)
```

# What AST Gives You

Instead of:

random text chunks

You get:

real code structure

---

## Phase 5 — Build Full Search Pipeline

Now combine everything.

---

## Final MVP Flow

```
Repository
↓
Read Files
↓
Chunk Code
↓
Generate Embeddings
↓
Store in FAISS
↓
User Query
↓
Convert Query to Embedding
↓
Search Similar Vectors
↓
Return Matching Code
```

# Example Final Workflow

## Repository Function

```
def verify_access_token(token):
```

## User Query

Where is authentication handled?

## System Behavior

1. Query becomes embedding
2. Function embedding already exists
3. Vector search finds similar vectors
4. Function returned as result

## Phase 6 — Build Frontend

Use:

## Streamlit

Install:

```
pip install streamlit
```

## Basic UI Example

```
import streamlit as st

st.title("Semantic Code Search")
```

```
query = st.text_input("Search codebase")

if query:
    st.write(query)
```

## Suggested Features

| Feature             | Difficulty |
|---------------------|------------|
| Search bar          | Easy       |
| Upload repo         | Medium     |
| Result cards        | Easy       |
| Syntax highlighting | Medium     |
| File navigation     | Medium     |

## Phase 7 — Add RAG

Only after search works properly.

## RAG Pipeline

```
User Question
    ↓
Retrieve Relevant Chunks
    ↓
Send Chunks to LLM
    ↓
Generate Explanation
```

## Example

User:

Explain authentication flow

System retrieves:

- login()
- verify\_token()
- middleware()

Then LLM explains the relationships.

---

## Optional LLM Choices

| Option     | Type  |
|------------|-------|
| OpenAI API | Cloud |
| Ollama     | Local |
| LM Studio  | Local |

---

## Important Beginner Advice

### DO NOT

- learn everything before building
- watch endless tutorials
- optimize architecture too early
- jump into AI agents immediately

### DO

- build tiny features daily
  - experiment constantly
  - debug patiently
  - learn concepts while building
-

# Recommended Learning Strategy

For every concept:

```
Learn 20 minutes
↓
Build tiny demo
↓
Experiment
↓
Repeat
```

This is the fastest way to learn AI engineering.

---

## First 14-Day Plan

### Days 1-2

Learn:

- Python file handling
- embeddings basics

Build:

- tiny semantic search
- 

### Days 3-4

Learn:

- cosine similarity
- vector search

Build:

- FAISS search
-

## Days 5–6

Learn:

- repository parsing
- chunking

Build:

- code chunk extraction
- 

## Days 7–8

Learn:

- embeddings pipeline

Build:

- semantic code retrieval
- 

## Days 9–10

Learn:

- Streamlit basics

Build:

- simple UI
- 

## Days 11–14

Improve:

- chunking
  - metadata
  - search accuracy
-

# Important Terms Cheat Sheet

| Term            | Meaning                          |
|-----------------|----------------------------------|
| Embedding       | Numerical meaning representation |
| Vector          | List of numbers                  |
| Semantic Search | Meaning-based search             |
| Chunk           | Small document/code piece        |
| Vector DB       | Database for embeddings          |
| Retrieval       | Finding relevant chunks          |
| RAG             | Retrieval + LLM generation       |
| AST             | Structured code representation   |
| Inference       | Using pretrained AI              |
| Fine-tuning     | Training AI further              |

---

## Recommended Resources

### Embeddings

Sentence Transformers: <https://www.sbert.net>

---

### Vector Search

FAISS: <https://github.com/facebookresearch/faiss>

---

### Streamlit

<https://streamlit.io>

---

### AST

<https://docs.python.org/3/library/ast.html>

---

# Final Advice

Your first goal is NOT:

Build enterprise AI system

Your first goal is:

Search Python functions semantically

That small working system is already a real AI project.

Everything else grows from there.

Build incrementally. Learn while building. Do not wait for perfect understanding before starting.